



Writing the Standards Down

Lab 2.a — the governance documents an AI agent actually uses



Two hours, mostly hands-on

10 min	Setup + why	Where 1.b left us, and what standards buy you
25 min	The five artifacts	What each is — and how to tell them apart
20 min	How they get authored	When one is required, who reviews, drafting with Claude
25 min	Demo — write an ADR	The repository boundary, documented live
30 min	Derive + write an NFR	You read three real diffs and write the rule
15 min	Wire it in + review	Trigger table, binding line, smoke test

Timings are the plan, not a promise — say so, and keep us honest on them.



Where we start

Five minutes. Pair anyone who is broken before we start, not during the demo.

- **Your own copy.** The same one from 1.b — ``npm test`` green, ``npm run dev`` working.
- **Clean branch.** ``git checkout -b feat/governance`` before anything else.
- **Two examples already ship.** One ADR (``docs/adr/0001``) and one NFR (``docs/nfr/0001``).
- **Snippets arrive by copy-paste.** Your copy predates today's material — pull ``docs/labs`` with `giget`, or copy from GitHub raw.



Sixteen developers, sixteen private setups

The assessment called this a Cultural Gap: engineers ahead of shared norms.

WHAT WORKS TODAY

- Individual practice is genuinely strong
- Claude is productive on this repo
- The habits from 1.b landed

WHAT DOES NOT

- None of it is shared
- Every dev re-teaches Claude the same things
- No way to tell if a rule is being followed

WHAT TODAY CHANGES

- Habits become documents
- Documents get reviewed
- CLAUDE.md points at them

Alignment, not persuasion. Nobody here needs convincing that AI helps — the gap is that everyone solved it separately.



Generated context files made things worse.

An ETH Zurich study (Feb 2026) found LLM-generated context files lowered task success by about 3% versus no context at all, while raising cost more than 20%. Human-written files improved success by about 4%. Running `/init` and walking away is worse than doing nothing. You will use Claude to draft — from evidence in the code — and you will own every decision.



Collective, Project, Feature

Three layers. Today you work in the middle one and hear about the outer one.

COLLECTIVE

- Org-wide — every repo
- Standards, templates, guardrails
- Most teams have none

PROJECT

- This repo
- CLAUDE.md, ADRs, NFRs, recipes
- Most teams have an accident here

FEATURE

- This slice of work
- Path-scoped rules, feature specs
- Arrives just in time



Five types — and the question each answers

If you can name the question, you know which document you are writing.

Artifact	The question it answers	What it is
CLAUDE.md	What must the agent know before anything else?	Always-loaded context + pointers
ADR	WHY is it like this?	One decision, with its cost
NFR	What must always be TRUE — and how do we check?	A standing, testable rule
Best-practice recipe	HOW do I do X here?	A worked procedure
Rules file	What applies ONLY in these files?	Path-scoped context

That is the whole trick. Everything else on the next two slides is detail.



One subject, five artifacts

This repo has a real rule: only repositories/ may import db/store.ts. Here is what each artifact says about that same subject.

Artifact	What it says about the repository boundary
CLAUDE.md	One line, no rationale: "never access db.json outside repositories/". Always loaded
ADR	Why a repository layer was chosen, what else was considered, what it costs
NFR	The rule as a standing requirement — plus how compliance is checked
Best practice	adding-a-resource.md — the steps for adding a resource within that boundary
Rules file	Guidance that loads only when you are in the files it applies to

None of them is redundant. ADR = history. NFR = law. Recipe = procedure. CLAUDE.md = index. Rules file = context that arrives just in time.



Telling them apart when it is unclear

	CLAUDE.md	Rules file	ADR	NFR	Recipe
Answers	What to know first	What applies here	Why	What must hold	How
Tense	Present, standing	Present, scoped	Past — a choice made	Present — holds now	Imperative
Lifecycle	Revised continuously	Revised in place	Superseded, never edited	Revised in place	Updated freely
Carries a check?	No	No	No	YES — that is why	No
When it loads	ALWAYS	On matching paths	On demand	On demand + binding line	On demand
Lose it and...	The agent is unoriented	Guidance stops arriving	You lose the why	You lose the rule	You lose the how

Two rows do the work: "carries a check" isolates the NFR — the only one obliged to say how compliance is verified, which is why it leads into next week. "When it loads" separates the two context files from the three documents.



How big should it get?

The scaling answer, because "keep it short" stops being useful advice at some point.

THE HONEST ANSWER

- Lab 1.a said 100–200 lines. That is right — to start.
- Past that, do not grow the file. Add a trigger row.
- The table is the mechanism: always-loaded index, on-demand detail.
- At corpus scale (100+ docs) you stop copying and start serving them.

CLAUDE.MD, THIS REPO

Reference docs – read on demand

When you are...	Read first	
-----	-----	
Changing response	docs/adr/0001...	
shapes or codes		
Calling an	docs/nfr/0001...	
external API		

a trigger is a SITUATION,
not a topic



Two rungs, and when to graduate

This repo ships the light form. Know the heavier one exists and what it buys.

	Nygard (start here)	MADR (graduate to)
Sections	Context · Decision · Status · Consequences	+ Decision Drivers · Considered Options
Good for	Most decisions, most teams	Contested or expensive choices
Cost	Ten minutes	A meeting
Graduate when	Consequences keep coming out thin	You need the alternatives on record

The published failure mode is not template choice — it is that nobody defined when an ADR is required, who reviews it, or where it lives. That is slide 13.



Two rungs — and the field that matters

An NFR without a verification method is a wish with a document number.

	Prose NFR (this repo)	The enforceable form
Shape	A few paragraphs of rule	Classification · Requirements · Thresholds · Verification
Says what	Yes	Yes, as numbered MUSTs
Says how checked	Not really	Explicitly, per clause
Graduate when	The rule starts being ignored	You want a gate, not a hope

"How is this checked?" is the question that turns a rule into a gate — and it is the one you will be asked for in the exercise.



Recipes and rules files

Both are cheap, both are underused, and slides 7–8 already told them apart.

BEST-PRACTICE RECIPE

- Write it the second time you answer the same question
- Prescriptive steps, not principles
- This repo: adding-a-resource.md

RULES FILE

- ``.claude/rules/*.md``, scoped by path
- Loads only when those files are open
- This repo: testing.md on test files

If a rule only matters in some files, a rules file beats a CLAUDE.md line — it costs nothing until it is relevant.



When is a document required?

The reason ADR practices die is not the template. It is that nobody wrote down the trigger.

Write a...	When...
ADR	the choice is hard to reverse, the team disagreed, or a newcomer would ask "why is it like this?"
NFR	the rule spans features AND you can say how compliance is checked
Recipe	you have now answered the same question twice
CLAUDE.md line	the agent needs it every session — and it is one line, not a page
Rules file	it only applies when specific files are open

Undefined triggers are the documented cause of governance decay: no rule for when one is required, who reviews it, or where it lives.



Who writes, who reviews, where it lives

The operational half. This is what the "how we use these" README exists to answer.

- **Authored by a few.** Standards are written by a small group and adopted by many. That is normal, not elitist.
- **Reviewed by a name.** Peer review via PR is the acceptance gate — an unreviewed standard is a draft.
- **Lives where people already look.** In the repo, next to the code, pointed at from CLAUDE.md. Not a wiki nobody opens.
- **Owned after today.** Someone keeps it true. Unowned documents rot faster than no documents.



Drafting with Claude, honestly

The ETH finding again: generated context hurts, human-written helps. Draft, do not delegate.

THE DIVISION OF LABOUR

- Claude drafts from EVIDENCE in the code.
- You own the Decision and the Consequences.
- Never ask it what the team should do.
- Read the draft as someone who disagrees.

PROMPT STARTERS

```
# gather evidence
Read @server/src/repositories/ and
@server/src/services/todos.service.ts.
What boundary do these enforce, and
what breaks if violated? Do not
propose changes.
```

```
# draft, but leave the decision
Using @docs/adr/template.md, draft
an ADR for that boundary. Fill
Context and Consequences from what
you read. Leave Decision as a
question for me to answer.
```



Keeping them true

The half of governance nobody plans for: what happens when the world moves.

- **Supersede, never edit, an accepted ADR.** The old decision happened. Write a new one that replaces it.
- **Revise NFRs in place.** They describe what holds now, so they change when the rule changes.
- **Every rule cites the incident that produced it.** A "Why" line naming a real failure survives staff turnover; "best practice" does not.
- **Review on a cadence, not on vibes.** A date on the document, and a person against it.



Demo — write an ADR

The repository boundary. Watch the whole shape, then you drive.



The gap we are documenting

A real, load-bearing boundary in this repo with no recorded reasoning behind it.

- **The rule exists.** CLAUDE.md says routes and services never import ``db/store.ts``.
- **The code obeys it.** Only ``server/src/repositories/`` touches the store.
- **Nothing says why.** No ADR. A newcomer sees a constraint with no rationale — so they route around it.
- **Nothing checks it.** Which is next week's problem, and worth noticing now.



Evidence in, decision mine

Ten minutes, live. The interesting part is the Consequences section.

WHAT I AM DOING

- Prompt Claude to READ, not to conclude.
- Paste its Context. Edit it for accuracy.
- Write the Decision myself, in one sentence.
- Add a consequence that is genuinely a cost.

DOCS/ADR/0002-...MD

ADR-0002: Repository-only store access

Status

Accepted

Context

<from Claude, edited>

Decision

Only modules under
server/src/repositories/ may import
db/store.ts. <-- I wrote this

Consequences

- Good: persistence is swappable
- Bad: one more hop for simple reads



Checkpoint 1

NOBODY ADVANCES UNTIL THIS IS TRUE

You have docs/adr/0002-*.md staged, with all four sections filled — and a Consequences section containing at least one real cost.

Thumbs up, or say so. If your Decision line came out of Claude rather than out of you, rewrite it now — that is the one part of an ADR that cannot be delegated.



Your turn — derive a rule

Three real fixes from last week. You decide what the standard is.



Three fixes from last week

All three closed their issue. All three were reviewed. All three are green.

Fix	What shipped	CI
A	The bug is gone. No test was added.	green
B	A test was added. It passes — and it passed before the fix too.	green
C	A test was added that failed first, then passed after the fix.	green

Read them cold. No commentary from the front of the room yet.



Which of these are acceptable? Write the rule.

In your group: produce candidate clauses. For each one, tag it MECHANICAL (a script could decide it from the diff) or FUZZY (it needs a person). Do not resolve your disagreements too early — where you disagree is where the standard is actually being made.



Where we landed

Written down before anyone moves on. This is the agreed list, not my list.

MECHANICAL

- A change under server/src ships with a test
- The test file is co-located
- Full suite green

FUZZY

- The test failed first
- The test asserts behaviour, not implementation
- The assertion is meaningful

DECIDE NOW

- Accepted risk?
- Human gate?
- Or dropped?

"The test must have failed first" is the clause that matters most and the one no gate can decide. That is not a flaw in your rule — it is the finding.



What a mature team wrote

A real NFR from a mature 137-document corpus — shown now that your list exists, not before.

Section	What it contains
Classification	Category, priority, and which external framework it traces to
Requirements	Numbered MUST clauses — no prose, no hedging
Thresholds	Target / Warning / Critical, so "compliant" is measurable
Verification	Per clause: the exact test or check that proves it
Plan reference	Which phase implements it, so it is not aspirational

What did you catch that they did not? Their form is heavier than you need today; the Verification section is the part worth stealing.



Turn the clauses into an NFR

Twenty minutes. One doc in docs/nfr/, peer-reviewed by whoever is sitting next to you.

- **Copy the form of NFR-0001.** Same headings, same tone. You are matching a pattern, not inventing one.
- **Every clause says how it is checked.** Even if the answer is "a human, at review."
- **Mark the fuzzy ones.** Explicitly. An unmarked unenforceable clause is the dangerous kind.
- **Record a decision for each.** Accepted risk, human gate, or dropped — with a name against it.



Checkpoint 2

NOBODY ADVANCES UNTIL THIS IS TRUE

Your NFR names how each clause is checked — and which ones cannot be.

Artifact check, not a state check: I am reading your document, not your test output. A clause with no check and no mark is the one thing that fails this checkpoint.



A document nothing points at is a document nobody reads

Claude included. This is the step that makes the other two hours matter.

THREE EDITS TO CLAUDE.MD

- One trigger row per new doc.
- Phrase the trigger as a SITUATION you are in.
- One binding line for the NFR, so it applies unfetched.
- Then smoke-test in a FRESH session.

CLAUDE.MD

When you are...	Read first	
-----	-----	
Changing the store	docs/adr/0002...	
boundary		
Fixing a bug or	docs/nfr/0002...	
adding behaviour		

Do / Don't

- Do: every change under server/src ships with a test (NFR-0002)

smoke test, fresh session:

"what is our rule about tests?"



What you built, and what is next

Ship checklist: committed · peer-reviewed · CI green · smoke test passed.

- **An ADR.** The boundary now has a recorded reason and an honest cost.
- **An NFR you derived.** With the checks named, and the unenforceable parts marked.
- **Both wired in and smoke-tested.** Claude reads them when the situation applies.
- **Next week: teeth.** You will make something refuse to violate the clause you just wrote — and decide what to do about the part that cannot be checked.